

Introduction

Every so often a LeetCode problem shows up wearing a costume. It looks like game theory — two players, turns, optimal play — and that costume scares people off before they even read the constraints. But underneath, Stone Game is a dynamic programming problem about tracking a *difference*, not a total. That single reframing is what separates people who can solve this in an interview from people who freeze up trying to simulate an entire game tree.

This problem matters because the pattern underneath it — interval DP, where you build answers for larger ranges out of answers for smaller ranges — shows up constantly. Stone Game is just the friendliest possible introduction to it. Interviewers like it because it tests whether you can find the right state to track, not whether you can write a simulation. Learn the shape here, once, properly, and you'll recognize it instantly the next time it wears a different costume.

Problem Overview

You're given a row of piles of stones, an even number of piles in total, with the total stone count guaranteed to be odd. Two players, Alice and Bob, alternate turns, with Alice always going first. On each turn, a player must take an entire pile from one end of the row — either the far left or the far right — never from the middle. Both players play optimally, meaning each one always makes the move that maximizes their own final stone count.

The question: assuming both players play as well as possible, does Alice end up with more stones than Bob? Since the total is odd, a tie is impossible — someone strictly wins.

Example

Take the piles `[5, 3, 4, 5]`. Alice moves first and can only choose between the leftmost pile (5) or the rightmost pile (5) — nothing in the middle is reachable yet. Whatever she picks, Bob then chooses optimally from what remains, and so on until all four piles are gone.

Working this out by hand is exactly what the DP table below does for us, and the answer comes out to Alice winning by exactly one stone. That "by exactly one stone" detail matters — it's not enough to guess who wins, you need a method that proves it and generalizes to piles like `[3, 7, 2, 3]` too.

Intuition

The instinct most people reach for first is: track Alice's total, track Bob's total, simulate every possible sequence of moves, and see who ends up ahead. That works, but it's slow, and worse, it obscures the actual structure of the problem.

Here's the insight that unlocks everything: the total number of stones on the table never changes, no matter who takes what. If you know the total, and you know the *difference* between Alice's stones and Bob's stones, you can recover both individual scores. So instead of tracking two numbers, track one: the gap.

This matters because "the gap" is a quantity that composes cleanly across subgames. If you know the best gap achievable on a smaller row of piles, you can compute the best gap on a slightly bigger row just by considering the two moves available at the ends. That composability is exactly what dynamic programming needs.

Brute Force Solution

Idea: At every turn, recursively try taking the left pile and try taking the right pile, letting the opponent do the same on whatever remains, and pick the branch that's best for the current player.

Algorithm: Define a recursive function over a range of piles `[i, j]`. Base case: one pile left, take it. Otherwise, try both ends and recurse on the smaller remaining range, keeping whichever choice yields the better outcome for the player whose turn it is.

Advantages: Conceptually simple, provably correct, easy to reason about.

Disadvantages: Without caching, this recomputes the same subranges over and over. Each turn branches into two more, so the number of paths explored grows exponentially with the number of piles.

Complexity: Time $O(2^n)$, Space $O(n)$ for the recursion stack. Fine for a handful of piles, hopeless once you hit dozens.

Optimal Solution

The fix is to notice that the brute force recursion only ever depends on a *range* `[i, j]` of remaining piles — and there are only n^2 possible ranges. So instead of recomputing a range every time we visit it, we fill in a table once.

Define `dp[i][j]` as: if only piles `i` through `j` are left on the table, and it's your turn, what's the best gap (your stones minus your opponent's) you can guarantee?

There are exactly two choices at any range:

- **Take the left pile** (`piles[i]`): you gain that pile's value, and the opponent then plays optimally on `[i+1, j]`. From your perspective, whatever gap they secure counts *against* you. So your resulting gap is `piles[i] - dp[i+1][j]`.
- **Take the right pile** (`piles[j]`): symmetric logic gives `piles[j] - dp[i][j-1]`.

A smart player takes whichever choice produces the bigger gap:

```
dp[i][j] = max(piles[i] - dp[i+1][j], piles[j] - dp[i][j-1])
```

The base case is a single pile: if only pile `i` remains, you take it, so `dp[i][i] = piles[i]`.

Because computing `dp[i][j]` needs `dp[i+1][j]` and `dp[i][j-1]` — both *smaller* ranges — you must fill the table by increasing slice length, not by row. Fill in all length-1 ranges, then all length-2 ranges, and so on, until you reach the full row `dp[0][n-1]`.

Slice length Ranges filled Depends on

1	<code>dp[i][i]</code>	nothing (base case)
2	<code>dp[i][i+1]</code>	two length-1 cells
3	<code>dp[i][i+2]</code>	two length-2 cells
...	...	...
n	<code>dp[0][n-1]</code>	the final answer

Python Code

```
def stoneGame(piles: list[int]) -> bool:
    n = len(piles)
    dp = [[0] * n for _ in range(n)]

    # Base case: a single pile — you just take it
    for i in range(n):
        dp[i][i] = piles[i]

    # Fill by increasing slice length so smaller ranges are ready first
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            take_left = piles[i] - dp[i + 1][j]
            take_right = piles[j] - dp[i][j - 1]
            dp[i][j] = max(take_left, take_right)
```

```
return dp[0][n - 1] > 0
```

Code Walkthrough

- `dp = [[0] * n for _ in range(n)]` builds an empty $n \times n$ grid — one cell per possible (i, j) range.
- The first loop fills the diagonal (`dp[i][i]`), the base case where only one pile remains.
- The outer `length` loop walks from 2 up to `n`, guaranteeing every range we compute only depends on already-filled smaller ranges.
- `j = i + length - 1` derives the end index of the current range from its start and length.
- `take_left` and `take_right` apply the recurrence directly: pile value minus the opponent's best achievable gap on what's left.
- `dp[i][j] = max(...)` records the optimal choice for that range.
- `dp[0][n-1] > 0` checks whether Alice, playing the full row first, can force a positive gap.

Dry Run

Piles: `[5, 3, 4, 5]`

Length 1 (base case): `dp[0][0]=5`, `dp[1][1]=3`, `dp[2][2]=4`, `dp[3][3]=5`

Length 2: - `dp[0][1]` (`[5,3]`): `left = 5 - dp[1][1] = 2`, `right = 3 - dp[0][0] = -2` → `2 - dp[1][2]` (`[3,4]`): `left = 3 - 4 = -1`, `right = 4 - 3 = 1` → `1` - `dp[2][3]` (`[4,5]`): `left = 4 - 5 = -1`, `right = 5 - 4 = 1` → `1`

Length 3: - `dp[0][2]` (`[5,3,4]`): `left = 5 - dp[1][2] = 4`, `right = 4 - dp[0][1] = 2` → `4 - dp[1][3]` (`[3,4,5]`): `left = 3 - dp[2][3] = 2`, `right = 5 - dp[1][2] = 4` → `4`

Length 4 (final): `dp[0][3] = max(5 - dp[1][3], 5 - dp[0][2]) = max(1, 1) = 1`

The final gap is `1`, which is positive — Alice wins by exactly one stone.

Complexity Analysis

Time: $O(n^2)$ — there are n^2 ranges (i, j) , and each is computed in $O(1)$ work.

Space: $O(n^2)$ for the table. This is correct because every range genuinely needs to be stored — smaller ranges are reused by multiple larger ranges, so recomputation isn't an option without the table.

Both bounds are tight: you can't do better than visiting every range once if you want the general interval-DP answer that works under arbitrary rules.

Alternative Solutions

There's a well-known $O(1)$ shortcut for *this specific version* of the problem. Piles split naturally into an even-indexed group (0, 2, 4, ...) and an odd-indexed group (1, 3, 5, ...). Alice can always force herself to take every pile in whichever group she prefers, simply by mirroring Bob's choice each turn. Since the total is guaranteed odd, the two groups can never be equal, so Alice always wins by picking the larger group upfront:

```
def stoneGameParity(piles: list[int]) -> bool:
    return True # always true given even pile count + odd total
```

This is faster, but it only holds because of this problem's exact guarantees (even pile count, odd total). Change those constraints — as later versions of Stone Game do — and the shortcut breaks entirely, while the DP still works unmodified. Know both: the DP is the transferable skill, the parity trick is a fun footnote specific to this exact problem.

Edge Cases

- **Two piles:** table is trivial; Alice takes the bigger one, and since the total is odd, they can never be equal.
- **Duplicate values:** the recurrence handles this fine — ties in raw pile value don't create ties in the game outcome, since position still matters.
- **Maximum input (500 piles):** the $O(n^2)$ table has about 250,000 cells, which still runs instantly.
- **All piles nearly equal:** the gap may be small, but since the total is odd, it can never be exactly zero.

Common Mistakes

1. **Filling the table in row order instead of by increasing length.** The recurrence needs `dp[i+1][j]` and `dp[i][j-1]` already computed — filling row by row reads cells that are still zero.
2. **Tracking each player's raw stone total instead of the gap.** The subtraction trick only works when you track the difference, not absolute scores.

3. **Writing `range(2, n)` instead of `range(2, n + 1)`**. This silently skips computing the full-row answer and gives a wrong result with no error.
4. **Forgetting to negate the opponent's gap**. The recurrence subtracts the opponent's best result because their gain is your loss — skipping the subtraction inflates every value.
5. **Assuming the parity shortcut always applies**. It only holds for even pile counts and odd totals; using it on a modified version of the problem gives a wrong answer.

Interview Questions

- What if the game required taking exactly two piles at a time?
- Can you solve this with memoized recursion instead of a bottom-up table? Show both.
- How would you reduce the space complexity if only two players ever play (hint: 1D rolling array)?
- What changes if the total number of stones is allowed to be even?
- Can you prove why Alice always wins in this specific variant without coding it?

Similar Problems

- **Stone Game II / III** — same interval-DP shape, but the parity shortcut no longer applies, so the table becomes mandatory.
- **Predict the Winner** — nearly identical mechanics; a great problem to confirm you understand the pattern rather than memorized code.
- **Burst Balloons** — a harder interval DP problem using the same "smaller ranges first" table-filling technique.
- **Matrix Chain Multiplication** — the classic interval DP problem this pattern is modeled after.

Key Takeaways

- Reframe two competing totals as a single difference whenever possible — it often turns an intractable simulation into a clean recurrence.
- Interval DP problems must be filled by increasing range length, never by row or column order.
- A working brute force is still worth writing first — it's your correctness check before you trust the optimized table.
- Shortcuts that exploit a problem's specific constraints are worth knowing, but never as a substitute for the general technique.

Watch the Video

For the full visual walkthrough — including the table filling in live and the parity trick explained on-screen — watch the video here: {LINK}

About the Series

This is part of **Fun with Learning Technology**, a series that walks through LeetCode problems, data structures, and software engineering concepts the way you'd want a senior engineer to explain them to you: starting from intuition, not from code.

Call To Action

If this made interval DP click, do three things: like the video on YouTube so more people find it, subscribe so you don't miss the next problem in this series, and subscribe to the Substack for the written breakdowns like this one. Drop a comment with the pile array you traced by hand — I read every one. And if you know someone prepping for interviews, share this with them.

Quick Review

Two-pile prefix-sum game: when do you spot this pattern?

Whenever a game lets you pick from either end of a sequence and you must find optimal alternating play — think interval DP on (left,right).

Time complexity of the interval DP solution for Stone Game?

$O(n^2)$ — one state per (i,j) subarray pair, filled once.

Space complexity, and how to shrink it?

$O(n^2)$ for the full DP table; collapsible to $O(n)$ with a 1D rolling array over diagonals.

Trickiest bug when coding the gap/score-difference DP?

Mixing up whose turn it is — the recurrence tracks score DIFFERENCE (current player minus opponent), not raw totals, so signs flip each layer.

Stone Game DP vs plain 0/1 knapsack — what's the key difference?

Knapsack picks any subset once; Stone Game alternates two players picking from array ends, so state must model turn-optimal minimax, not simple inclusion.

Interview follow-up: does first player always win with even-length piles?

Yes if total stones is odd (classic Stone Game); first player can always force a win via parity/prefix-sum argument, no DP needed to prove it.